

Game Engine Documentation

Project by: Nafisa Anjum Neha (ID: 230041221)

Section: Scene & Utility Management

Description:

Implemented a modular Scene and Utility system for a small game engine. Focused on **OOP concepts**, including **inheritance**, **polymorphism**, **abstraction**, **templates**, and **exception handling**.

Scene System Files

Scene.cpp

- `Scene(const std::string& sceneName)` — Initializes scene with a name. Base class for polymorphic scene handling.
- `void update(float deltaTime)` — Updates the scene each frame. Empty in base class.
- `void addEntity(std::shared_ptr<Entity> entity)` — Adds an entity to the scene.
- `void removeEntity(std::shared_ptr<Entity> entity)` — Removes an entity from the scene.
- `std::string getName() const` — Returns the scene's name.
- `std::vector<std::shared_ptr<Entity>> getEntities() const` — Returns all entities in the scene.

SceneFactory.cpp

- `std::shared_ptr<Scene> createScene(const std::string& sceneName)` — Creates a scene by name (Menu, Game, Pause, Settings, Credits). Returns nullptr if unknown.

SceneManager.cpp

- `SceneManager()` — Constructor; initializes manager.
- `~SceneManager()` — Destructor; cleans up manager.
- `void addScene(const std::shared_ptr<Scene>& scene)` — Adds an existing scene.
- `void createAndAddScene(const std::string& sceneName)` — Creates and adds a new scene.
- `void removeScene(const std::string& sceneName)` — Removes a scene. Clears current scene if active.
- `void switchScene(const std::string& sceneName)` — Switches active scene; saves previous in history.
- `std::shared_ptr<Scene> getCurrentScene() const` — Returns the active scene.
- `void printAllScenes() const` — Prints all managed scenes.
- `void goBack()` — Returns to previous scene in history.

SettingsScene.cpp

- `SettingsScene()` — Constructor; initializes Settings scene, sets menu open.

- void openSettings() — Opens settings menu.
- void closeSettings() — Closes settings menu.
- void update(float deltaTime) — Updates scene; prints if settings are open or closed.

MenuScene.cpp

- MenuScene() — Constructor; initializes Menu scene.
- void update(float deltaTime) — Updates menu scene.

CreditsScene.cpp

- CreditsScene() — Constructor; initializes Credits scene.
- void update(float deltaTime) — Updates credits scene.

GameScene.cpp

- GameScene() — Constructor; initializes Game scene, sets isRunning=false.
- void startGame() — Starts game; resets elapsed time.
- void stopGame() — Stops game.
- void update(float deltaTime) — Updates game; increments elapsed time if running.

PauseScene.cpp

- PauseScene() — Constructor; initializes Pause scene, sets paused=true.
- void pause() — Pauses the game.
- void resume() — Resumes the game.
- void update(float deltaTime) — Updates pause state; prints paused or active status.

Utils System Files

ConfigManager.cpp

- static ConfigManager* GetInstance() — Singleton instance.
- void SetValue(const std::string& key, const std::string|int|float value) — Sets a config value.
- std::string GetValue(const std::string& key) const — Returns value as string.
- int GetValueInt(const std::string& key) const — Returns value as int.
- float GetValueFloat(const std::string& key) const — Returns value as float.
- void PrintAll() const — Prints all settings.

DebugAssert.h

- DEBUG_ASSERT(condition, message) — Checks a condition at runtime; prints message, file, line, and exits if false.

EventDispatcher.h

- **Class EventListener** — Base class; must implement void OnEvent(const std::string& eventName).

- **Class EventDispatcher** — Registers/unregisters listeners and triggers events.

InputManager.h

- Abstract base class for input handling. Singleton-style active instance.
- virtual bool PausePressed() const = 0 — Returns if pause input is pressed.
- virtual bool StartPressed() const = 0 — Returns if start input is pressed.
- virtual bool KeyPressed(int key) const = 0 — Returns if key is pressed.
- static void SetActive(InputManager* manager) — Sets global active manager.
- static bool Pause() / Start() / Key(int key) — Calls respective function on active manager.

Logger.h

- **Class Logger** — Abstract base class for logging.
- virtual void log(const std::string& message, LogLevel level) — Must be implemented.
- **Class ConsoleLogger** — Logs to console.
- **Class FileLogger** — Logs to file.

ResourceManager.h

- Template class for shared resource management.
- void Load(const std::string& key, std::shared_ptr<T> resource) — Loads resource.
- std::shared_ptr<T> operator[](const std::string& key) — Access resource by key.
- void Unload(const std::string& key) — Removes resource.
- void PrintAll() const — Prints all loaded resources.

Timer.h

- **Class Timer** — Tracks elapsed time.
- update(float deltaTime) — Adds deltaTime.
- hasElapsed(float duration) — Checks if elapsed.
- reset() — Resets timer.
- float restart() — Returns current time and resets.
- **Class CountdownTimer : Timer** — Countdown timer.
- getRemaining() — Returns remaining time before countdown ends.